

МИНИСТЕРСТВО НАУКИ ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский Нижегородский государственный
университет им. Н. И. Лобачевского

Институт информационных технологий, математики и механики

УЧЕБНЫЙ ПРОЕКТ ПО ДИСЦИПЛИНЕ
«ТЕХНОЛОГИИ БАЗ ДАННЫХ»

на тему:

«Ювелирная мастерская»

Выполнил:

студентка группы 3824Б1ПР1
Шевелева Ксения Андреевна

Проверил:

доцент кафедры МОСТ, к.ф.-
м.н.
Шапошников Д.Е.

Нижний Новгород
2026

Описание предметной области

База данных разрабатывается для ювелирной мастерской, которая занимается изготовлением ювелирных изделий для частных лиц на заказ.

Основные сущности:

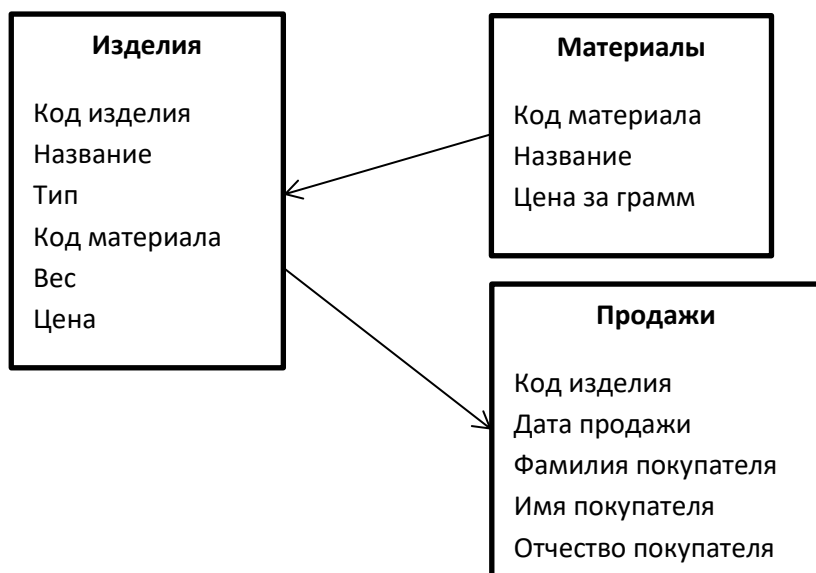
- Изделия – ювелирные украшения (кольца, серьги, броши, браслеты)
- Материалы – драгоценные металлы и камни
- Продажи – покупки изделия

Развитие постановки задачи:

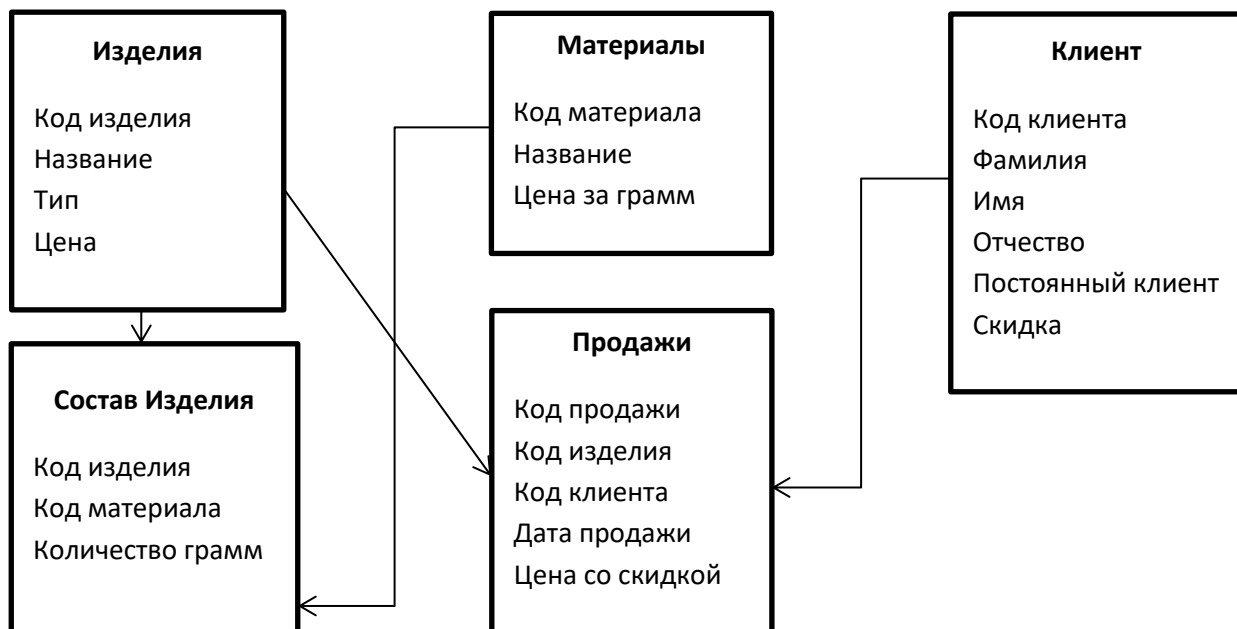
В исходной версии изделие состояло из одного материала. В финальной версии изделие может состоять из нескольких материалов, а также добавлена система скидок для постоянных клиентов.

1. Концептуальная схема

Исходная (без развития):



С развитием:



Реляционная структура

Исходная (без развития):

Таблица	Поле	Тип	Not NULL	PK/FK	CHECK
products	product_id	SERIAL	+	PK	
products	product_name	TEXT	+		
products	product_type	TEXT	+		product_type IN ('серьги','кольца','броши','браслеты')
products	material_id	INTEGER	+	FK	
products	weight	NUMERIC	+		weight>0
products	product_price	NUMERIC	+		product_price>0
materials	material_id	SERIAL	+	PK	
materials	material_name	TEXT	+		
materials	material_price	NUMERIC	+		material_price>0
sales	product_id	INTEGER	+	FK	
sales	sale_date	DATE	+		sale_date <= CURRENT_DATE
sales	last_name	TEXT	+		
sales	first_name	TEXT	+		
sales	middle_name	TEXT	-		

С развитием:

Таблица	Поле	Тип	Not NULL	PK/FK	CHECK
products	product_id	SERIAL	+	PK	
products	product_name	TEXT	+		
products	product_type	TEXT	+		product_type IN ('серьги','кольца','броши','браслеты')
products	product_price	NUMERIC	+		product_price>0
materials	material_id	SERIAL	+	PK	
materials	material_name	TEXT	+		
materials	material_price	NUMERIC	+		material_price>0
product_materials	product_id	INTEGER	+	FK	
product_materials	material_id	INTEGER	+	FK	
product_materials	grams	NUMERIC	+		grams>0
customers	customer_id	SERIAL	+	PK	
customers	last_name	TEXT	+		
customers	first_name	TEXT	+		
customers	middle_name	TEXT	-		
customers	is_permanent	BOOLEAN	+		DEFAULT FALSE
customers	discount	INTEGER	+		Discount BETWEEN 0 AND 100
sales	sale_id	SERIAL	+		
sales	product_id	INTEGER	+	FK	
sales	customer_id	INTEGER	+	FK	
sales	sale_date	DATE	+		sale_date <= CURRENT_DATE
sales	final_price	NUMERIC	+		final_price>=0

Далее все задания выполнены для базы данных с развитием.

2. Структура базы данных (с развитием)

Скрипт на языке SQL:

```
CREATE TABLE products (  
    product_id SERIAL PRIMARY KEY,  
    product_name TEXT NOT NULL,  
    product_type TEXT NOT NULL CHECK (product_type IN ('серьги', 'кольца',  
'броши', 'браслеты')),  
    product_price NUMERIC NOT NULL CHECK (product_price > 0)  
);  
CREATE TABLE materials (  
    material_id SERIAL PRIMARY KEY,  
    material_name TEXT NOT NULL,  
    material_price NUMERIC NOT NULL CHECK (material_price > 0)  
);  
CREATE TABLE product_materials (  
    product_id INTEGER NOT NULL REFERENCES products(product_id),  
    material_id INTEGER NOT NULL REFERENCES materials(material_id),  
    grams NUMERIC NOT NULL CHECK (grams > 0),  
    PRIMARY KEY (product_id, material_id)  
);  
CREATE TABLE customers (  
    customer_id SERIAL PRIMARY KEY,  
    last_name TEXT NOT NULL,  
    first_name TEXT NOT NULL,  
    middle_name TEXT,  
    is_permanent BOOLEAN NOT NULL DEFAULT FALSE,  
    discount INTEGER NOT NULL CHECK (discount BETWEEN 0 AND 100)  
);  
CREATE TABLE sales (  
    sale_id SERIAL PRIMARY KEY,  
    product_id INTEGER NOT NULL REFERENCES products(product_id),  
    customer_id INTEGER NOT NULL REFERENCES customers(customer_id),  
    sale_date DATE NOT NULL CHECK (sale_date <= CURRENT_DATE),  
    final_price NUMERIC NOT NULL CHECK (final_price >= 0)  
);
```

3. Операторы языка SQL

а. Соединение таблиц

Общая сумма покупок по каждому клиенту:

```
SELECT  
    c.customer_id, c.last_name, c.first_name, c.is_permanent,  
    COUNT(s.sale_id) AS count_purchases,  
    SUM(s.final_price) AS total_sum  
FROM customers c  
LEFT JOIN sales s ON c.customer_id = s.customer_id  
GROUP BY c.customer_id, c.last_name, c.first_name, c.is_permanent  
ORDER BY total_sum DESC NULLS LAST;
```

Пример вывода:

customer_id	last_name	first_name	is_permanent	count_purchases	total_sum
2	Петрова	Екатерина	t	2	92930.50000000000000000000000000
1	Иванов	Иван	t	2	77676.30000000000000000000000000
3	Сидоров	Алексей	f	1	72072.00000000000000000000000000
4	Козлова	Мария	t	1	51714.00000000000000000000000000
5	Смирнов	Дмитрий	f	1	37440.00000000000000000000000000
6	Федорова	Анна	t	1	34465.60000000000000000000000000

(6 строк)

б. Скалярные функции работы с датой/временем

Сколько дней прошло с момента продажи изделия:

```
SELECT
s.sale_id, p.product_name, c.last_name || ' ' || c.first_name AS customer,
TO_CHAR(s.sale_date, 'Day') AS day_of_the_week,
TO_CHAR(s.sale_date, 'Month') AS month,
EXTRACT(YEAR FROM s.sale_date) AS year,
CURRENT_DATE - s.sale_date AS days_from_sale
FROM sales s
JOIN products p ON s.product_id = p.product_id
JOIN customers c ON s.customer_id = c.customer_id
ORDER BY s.sale_date;
```

Пример вывода:

sale_id	product_name	customer	day_of_the_week	month	year	days_from_sale
2	Серьги "Ангелина"	Петрова Екатерина	Saturday	May	2025	378
3	Брошь "Павлин"	Сидоров Алексей	Thursday	May	2025	373
4	Браслет "Золотая нить"	Козлова Мария	Tuesday	May	2025	368
5	Кольцо "Изумрудная роса"	Смирнов Дмитрий	Sunday	May	2025	363
6	Серьги "Алые паруса"	Федорова Анна	Sunday	June	2025	356
8	Кольцо "Изумрудная роса"	Петрова Екатерина	Tuesday	June	2025	347

(6 строк)

с. Скалярные функции с текстовым поиском

Поиск пробы металла в названии материала:

```
SELECT
material_id, material_name AS material,
UPPER(SPLIT_PART(material_name, ' ', 1)) AS metal,
SUBSTRING(material_name FROM '[0-9]+') AS metal_sample
FROM materials
WHERE material_name ~ '[0-9]'
```

Пример вывода:

material_id	material	metal	metal_sample
1	Платина 950	ПЛАТИНА	950
2	Золото 585	ЗОЛОТО	585
3	Золото 750	ЗОЛОТО	750
4	Серебро 925	СЕРЕБРО	925
5	Бриллиант 1кл	БРИЛЛИАНТ	1

(5 строк)

d. Поиск родительских записей, у которых нет дочерних

Клиенты без покупок изделий:

```
SELECT
c.customer_id, c.last_name, c.first_name, c.is_permanent, c.discount FROM
customers c
LEFT JOIN sales s ON c.customer_id = s.customer_id
WHERE s.sale_id IS NULL;
```

Пример вывода: (все клиенты хотя бы раз приобретали изделия)

```
customer_id | last_name | first_name | is_permanent | discount
-----+-----+-----+-----+-----
(0 строк)
```

4. Триггер INSERT для проверки правильности данных

Триггер на таблицу sales. Условие: при вставке продажи проверяется, что final_price соответствует расчетной цене с учетом скидки.

Триггер функция:

```
DECLARE
    full_price NUMERIC;
    customer_discount INTEGER;
    expected_price NUMERIC;
BEGIN
    SELECT product_price INTO full_price
    FROM products
    WHERE product_id = NEW.product_id;

    SELECT discount INTO customer_discount
    FROM customers
    WHERE customer_id = NEW.customer_id;

    expected_price := full_price * (1 - customer_discount / 100.0);

    IF NEW.final_price <> expected_price THEN
        RAISE EXCEPTION 'Ошибка: цена со скидкой должна быть %, а не %',
expected_price, NEW.final_price;
    END IF;
    RETURN NEW;
END;
```

Создание триггера:

```
CREATE TRIGGER trg_check_sale_price
BEFORE INSERT ON sales
FOR EACH ROW
EXECUTE FUNCTION check_sale_price();
```

Пример вывода:

```
jewelry_db=# INSERT INTO sales (product_id, customer_id, sale_date, final_price)
jewelry_db=# VALUES (1, 1, CURRENT_DATE, 5000);
ОШИБКА:  Ошибка: цена со скидкой должна быть 38902.50000000000000000000000000, а не 5000
КОНТЕКСТ:  функция PL/pgSQL check_sale_price(), строка 19, оператор RAISE
```

```
jewelry_db=# INSERT INTO sales (product_id, customer_id, sale_date, final_price)
jewelry_db=# VALUES (1, 1, CURRENT_DATE, 38902.50);
INSERT 0 1
```

5. Процедура с курсором для модификации определенных записей по условию

Условие: для постоянных клиентов со скидкой менее 15% повысить скидку на 5%, если они совершили более 2 покупок.

Создание процедуры:

```
CREATE OR REPLACE PROCEDURE increase_discount_for_loyal_customers()
LANGUAGE plpgsql
AS $$
DECLARE
    cur_customers CURSOR FOR
        SELECT customer_id, last_name, first_name, discount
        FROM customers
        WHERE is_permanent = TRUE AND discount < 15
        FOR UPDATE;

    v_customer_id INTEGER;
    v_last_name TEXT;
    v_first_name TEXT;
    v_discount INTEGER;
    v_purchase_count INTEGER;
    v_new_discount INTEGER;
BEGIN
    OPEN cur_customers;
    LOOP
        FETCH cur_customers INTO v_customer_id, v_last_name, v_first_name,
v_discount;
        EXIT WHEN NOT FOUND;

        SELECT COUNT(*) INTO v_purchase_count
        FROM sales
        WHERE customer_id = v_customer_id;

        IF v_purchase_count > 2 THEN
            v_new_discount := v_discount + 5;
            RAISE NOTICE 'Клиенту % % (скидка была %) повышаем скидку до %
(покупок: %)', v_first_name, v_last_name, v_discount, v_new_discount,
v_purchase_count;

            UPDATE customers
            SET discount = v_new_discount
            WHERE CURRENT OF cur_customers;
        ELSE
            RAISE NOTICE 'Клиент % % (скидка %) - пропускаем, покупок: %
(нужно >2)', v_first_name, v_last_name, v_discount, v_purchase_count;
        END IF;
    END LOOP;
    CLOSE cur_customers;
    RAISE NOTICE 'Процедура завершена.';
END;
$$;
```

Вызов процедуры:

```
CALL increase_discount_for_loyal_customers();
```

Пример вывода:

```
ЗМЕЧАНИЕ: Клиенту Иван Иванов (скидка была 10) повышаем скидку до 15 (покупок: 3)
ЗМЕЧАНИЕ: Клиент Мария Козлова (скидка 10) – пропускаем, покупок: 1 (нужно >2)
ЗМЕЧАНИЕ: Процедура завершена.
```

6. Процедура для удаления родительской записи с дочерними

Условие: удалить клиента и все его покупки.

Создание процедуры:

```
CREATE OR REPLACE PROCEDURE delete_customer_with_sales(p_customer_id INTEGER)
LANGUAGE plpgsql
AS $$
DECLARE
    v_customer_exists BOOLEAN;
    v_sales_count INTEGER;
BEGIN
    SELECT EXISTS (
        SELECT 1 FROM customers WHERE customer_id = p_customer_id
    ) INTO v_customer_exists;

    IF NOT v_customer_exists THEN
        RAISE EXCEPTION 'Клиент с ID % не найден', p_customer_id;
    END IF;

    SELECT COUNT(*) INTO v_sales_count
    FROM sales
    WHERE customer_id = p_customer_id;

    DELETE FROM sales
    WHERE customer_id = p_customer_id;

    RAISE NOTICE 'Удалено % продаж(а) клиента с ID %', v_sales_count,
p_customer_id;

    DELETE FROM customers
    WHERE customer_id = p_customer_id;

    RAISE NOTICE 'Клиент с ID % успешно удалён', p_customer_id;

EXCEPTION
    WHEN OTHERS THEN
        RAISE NOTICE 'Ошибка при удалении клиента %: %', p_customer_id,
SQLERRM;
        RAISE;
END;
$$;
```

Пример вывода:

```
ЗМЕЧАНИЕ: Удалено 3 продаж(а) клиента с ID 1
ЗМЕЧАНИЕ: Клиент с ID 1 успешно удалён
```


7. Схема EAV необязательных атрибутов и функции

Создание таблицы для хранения дополнительных атрибутов изделий:

```
CREATE TABLE product_attributes (  
    attr_id SERIAL PRIMARY KEY,  
    attr_name TEXT UNIQUE NOT NULL  
);
```

Создание таблицы значений атрибутов:

```
CREATE TABLE product_attr_values (  
    product_id INTEGER REFERENCES products(product_id) ON DELETE CASCADE,  
    attr_id INTEGER REFERENCES product_attributes(attr_id) ON DELETE CASCADE,  
    attr_value TEXT NOT NULL,  
    PRIMARY KEY (product_id, attr_id)  
);
```

Функция добавления нового атрибута:

```
CREATE OR REPLACE FUNCTION add_product_attribute(  
    p_attr_name TEXT  
)  
RETURNS TEXT AS $$  
BEGIN  
    INSERT INTO product_attributes (attr_name)  
    VALUES (p_attr_name)  
    ON CONFLICT (attr_name) DO NOTHING;  
  
    RETURN 'Атрибут "' || p_attr_name || '" добавлен';  
END;  
$$ LANGUAGE plpgsql;
```

Функция добавления значения атрибута:

```
CREATE OR REPLACE FUNCTION set_product_attribute(  
    p_product_id INTEGER,  
    p_attr_name TEXT,  
    p_attr_value TEXT  
)  
RETURNS TEXT AS $$  
DECLARE  
    v_attr_id INTEGER;  
    v_product_exists BOOLEAN;  
BEGIN  
  
    SELECT EXISTS(SELECT 1 FROM products WHERE product_id = p_product_id)  
    INTO v_product_exists;  
  
    IF NOT v_product_exists THEN  
        RETURN 'Ошибка: изделие с ID ' || p_product_id || ' не найдено';  
    END IF;  
  
    SELECT attr_id INTO v_attr_id  
    FROM product_attributes  
    WHERE attr_name = p_attr_name;  
  
    IF v_attr_id IS NULL THEN  
        RETURN 'Ошибка: атрибут "' || p_attr_name || '" не найден';  
    END IF;  
  
    INSERT INTO product_attr_values (product_id, attr_id, attr_value)  
    VALUES (p_product_id, v_attr_id, p_attr_value)  
    ON CONFLICT (product_id, attr_id)  
    DO UPDATE SET attr_value = EXCLUDED.attr_value;
```

```

        RETURN 'Атрибут "' || p_attr_name || '" = "' || p_attr_value || '"
добавлен изделию ' || p_product_id;
END;
$$ LANGUAGE plpgsql;

```

Примеры добавления атрибутов и их значений:

```

jewelry_db=# SELECT add_product_attribute('размер_кольца');
add_product_attribute
-----
Атрибут "размер_кольца" добавлен
(1 строка)

```

```

jewelry_db=# SELECT set_product_attribute(1, 'размер_кольца', '17.5');
set_product_attribute
-----
Атрибут "размер_кольца" = "17.5" добавлен изделию 1
(1 строка)

```

```

jewelry_db=# SELECT set_product_attribute(1, 'цвет', 'золотой');
set_product_attribute
-----
Ошибка: атрибут "цвет" не найден
(1 строка)

```

Вывод таблицы со всеми атрибутами:

```

CREATE VIEW product_attributes_view AS
SELECT
    p.product_id,
    p.product_name,
    p.product_type,
    a.attr_name,
    v.attr_value
FROM products p
LEFT JOIN product_attr_values v ON p.product_id = v.product_id
LEFT JOIN product_attributes a ON v.attr_id = a.attr_id
ORDER BY p.product_id, a.attr_name;

```

```

jewelry_db=# SELECT * FROM product_attributes_view;
product_id | product_name | product_type | attr_name | attr_value
-----+-----+-----+-----+-----
1 | Кольцо "Классика" | кольца | размер_кольца | 17.5
2 | Серьги "Ангелина" | серьги | тип_застежки | английская
3 | Брошь "Павлин" | броши |
4 | Браслет "Золотая нить" | браслеты | длина_браслета | 19 см
5 | Кольцо "Изумрудная роса" | кольца | размер_кольца | 16.5
6 | Серьги "Алые паруса" | серьги |
(6 строк)

```

8. Аналитический набор с рейтингом

Рейтинг клиентов по сумме покупок и опережение от следующего.

Создание аналитической таблицы (как представление):

```
CREATE OR REPLACE VIEW customer_rating_analytics AS
WITH customer_totals AS (
    SELECT
        c.customer_id,
        c.last_name,
        c.first_name,
        c.is_permanent,
        c.discount,
        COALESCE(ROUND(SUM(s.final_price), 2), 0) AS total_spent,
        COUNT(s.sale_id) AS purchase_count
    FROM customers c
    LEFT JOIN sales s ON c.customer_id = s.customer_id
    GROUP BY c.customer_id, c.last_name, c.first_name, c.is_permanent,
    c.discount
),
ranked AS (
    SELECT *,
        RANK() OVER (ORDER BY total_spent DESC) AS rating,
        LEAD(total_spent) OVER (ORDER BY total_spent DESC) AS next_total
    FROM customer_totals
)
SELECT
    customer_id,
    last_name,
    first_name,
    is_permanent,
    discount,
    purchase_count,
    total_spent,
    rating,
    CASE
        WHEN next_total IS NULL THEN 0
        ELSE total_spent - next_total
    END AS lead_to_next
FROM ranked
ORDER BY rating;
```

Создание таблицы из представления:

```
CREATE TABLE customer_rating_table AS
SELECT * FROM customer_rating_analytics;
```

Пример вывода:

customer_id	last_name	first_name	is_permanent	discount	purchase_count	total_spent	rating	lead_to_next
2	Петрова	Екатерина	t	15	2	92930.50	1	20858.50
3	Сидоров	Алексей	f	0	1	72072.00	2	20358.00
4	Козлова	Мария	t	10	1	51714.00	3	14274.00
5	Смирнов	Дмитрий	f	0	1	37440.00	4	2974.40
6	Федорова	Анна	t	20	1	34465.60	5	0

(5 строк)